Kosmocrates Production Substrate Operationalization Specification v0.1

Runtime Continuation Delta from the Implemented `kosmo-*` Baseline

KOSMO-OPS-01

This document specifies the next implementation layer after the initial `kosmo-*` production substrate has been implemented. It is intended as a direct handoff to a coding agent. It does not re-specify HYPHAE, Metatron, LPCM or SystemCube from first principles. It defines how to operationalize the implemented baseline through real Foundry execution, parse-back validation, persistent CorpusCartography, a PSE bridge, isolated materialization, deterministic SystemCube export, controlled acquisition and empirical evaluation.

Document status: implementation-target specification
Source of truth: current repository baseline plus `SUBSTRATE.md`, `PHASE_CHECKLIST.md`, `IMPLEMENTATION_STATUS.md`, `SPEC_TRACEABILITY.md`, `SAFETY_POLICY.md`

Generated for coding-agent continuation work

Contents

Front Matter

Normative language

The keywords **MUST**, **MUST NOT**, **SHOULD**, **MAY**and **SHALL** are normative. Rules labeled with stable identifiers are implementation requirements. A coding agent implementing this document must preserve all baseline invariants unless a later explicitly approved migration document supersedes them.

Source of truth

The previous HYPHAE, Workbench, Metatron, LPCM and SystemCube specification corpus has been consumed into the implemented `kosmo-*` production substrate. From this point forward, the primary implementation source of truth is the current repository baseline, not the historical PDFs.

The implementation baseline consists of the following crates:

```
kosmo-core
kosmo-workbench
kosmo-hyphae
kosmo-systemcube
kosmo-pipeline
```

Future implementation agents must inspect the existing APIs, tests and control documents before coding. The historical specifications remain design context, but they must not be used to create parallel or duplicate implementations.

Central thesis

```
The previous specification corpus has been consumed into the implemented
kosmo-* production substrate.

The next implementation phase MUST NOT rebuild the consumed corpus.

It MUST extend the existing crates from governance skeletons and planning
artifacts into validated runtime capabilities, while preserving current
invariants and all existing tests.
```

Reading order for a coding agent

1. Read `SUBSTRATE.md`.
2. Read `PHASE_CHECKLIST.md`.
3. Read `IMPLEMENTATION_STATUS.md`.
4. Read `SPEC_TRACEABILITY.md`.
5. Read `SAFETY_POLICY.md`.
6. Inspect `crates/kosmo-core`, `crates/kosmo-workbench`, `crates/kosmo-hyphae`, `crates/kosmo-systemc` and `crates/kosmo-pipeline`.

7. Run the baseline tests.
8. Only then implement the next operationalization phase.

Part I — Implemented Baseline Contract

1.1 Purpose

This part locks the implemented `kosmo-*` crates as the baseline for all future work. The goal is to prevent reimplementation, duplicated concepts, policy weakening and accidental bypass of already-tested safety boundaries.

1.2 Current crate map

Crate	Baseline role
`kosmo-core`	Substrate types: `Digest`, `Q16`, `PolicyProfile`, `EvidenceBundle`, `GateResult`, authority/taint/license labels, capability locks, run descriptors and Foundry result types.
`kosmo-workbench`	Workspace scanning, `TaskSpec`, `ContextPack`, Foundry runner skeleton and structured run reports.
`kosmo-hyphae`	HYPHAE v0.3/v0.4, CubeSwarm, CorpusCartography, Metatron microtopology, planning-only Surgery and LPCM passive report.
`kosmo-systemcube`	SystemCube, BlueprintUnit, SystemCubeManifest, contradiction energy, compatibility, D-density and dry-run `KcubeExportReport`.
`kosmo-pipeline`	Unified dry-run pipeline, gate trace aggregation, integration reports, materialization governance skeleton and operator approval token path.

1.3 Verification baseline

The authoritative verification baseline for this continuation specification is:

```
kosmo-core:        49 passed
kosmo-workbench:   20 passed
kosmo-hyphae:     127 passed
kosmo-systemcube:  36 passed
kosmo-pipeline:    46 passed

TOTAL:            278 passed, 0 failed, 0 warnings
```

Older intermediate counts in status documents must be treated as stale progress counters. A continuation phase may update the count only after running and recording a full verification.

1.4 Baseline rules

BASELINE-001: The `kosmo-*` crates are the canonical implementation baseline.

BASELINE-002: Future agents **MUST**inspect existing public APIs, tests and control files before implementing any continuation layer.

BASELINE-003: Future agents **MUST NOT**re-create Phase 0–11 from the old PDFs.

BASELINE-004: Future agents **MAY**refactor only if existing tests remain green and public semantic contracts are preserved.

BASELINE-005: Any new feature **MUST**add tests without weakening existing tests.

1.5 Hard non-reimplementation rules

NONREIMPL-001: Do not create a second `Digest` type.

NONREIMPL-002: Do not create a second `PolicyProfile`.

NONREIMPL-003: Do not create a second `EvidenceBundle` model.

NONREIMPL-004: Do not create parallel `GateResult` or gate-merge semantics.

NONREIMPL-005: Do not create a second HostCube / VoidMap / DeficiencyVector pipeline unless it is an explicit adapter over the existing one.

NONREIMPL-006: Do not create a second `CorpusCartography`.

NONREIMPL-007: Do not create a second Metatron microtopology stack.

NONREIMPL-008: Do not create a second SystemCube export model.

NONREIMPL-009: Do not create a second pipeline runner.

NONREIMPL-010: Do not implement direct materialization paths outside `MaterializationPlan` and its explicitly specified successors.

1.6 Invariants

The following invariants are not aspirational. They are baseline behavior and must survive all continuation phases:

- Content-addressing everywhere: durable records use deterministic `Digest` over canonical content fields.
- `Q16` fixed-point arithmetic in gate, audit, policy, metric and digest-relevant paths.
- Default `PolicyProfile` is `ReportOnly`.
- Fail-closed policy checks.
- Evidence-bound durable objects.
- Deterministic replay.
- Sorted collections and `BTreeMap`-style deterministic ordering in content-addressed paths.
- Worst-wins gate aggregation: `Reject > Warn > Pass`.
- No host writes by default.
- No network acquisition by default.
- Policy consistency across all subreports.

INVARIANT-001: No `f32` or `f64` may enter audit, gate, digest, replay, policy or certificate paths.

INVARIANT-002: No `HashMap` or `HashSet` iteration may affect content-addressed output.

INVARIANT-003: Default policy remains `ReportOnly`.

INVARIANT-004: Any capability escalation requires explicit policy transition.

INVARIANT-005: Gate aggregation remains worst-wins.

INVARIANT-006: Every durable record **MUST**remain evidence-bound or explicitly non-durable.

INVARIANT-007: Identical inputs **MUST**produce identical report IDs.

INVARIANT-008: Metric saturation **MUST NOT**bypass policy.

1.7 Layer contracts

1.7.1 `kosmo-core`

All future crates must use `kosmo-core` primitives where applicable. New durable objects must use the existing digest model. Policy checks must route through `PolicyProfile` instead of local ad-hoc booleans.

1.7.2 `kosmo-workbench`

Workbench remains the intended path toward execution. HYPHAE, Metatron, LPCM and SystemCube must not execute. Real Foundry execution must be implemented in Workbench or a Workbench-owned runtime boundary.

1.7.3 `kosmo-hyphae`

HYPHAE remains an assimilation, diagnostic, planning and evidence layer. It must not become a patching engine. Rejected structural yields must continue producing negative evidence.

1.7.4 `kosmo-systemcube`

SystemCube export remains blueprint/package export. D-density is diagnostic, not authorization. `KcubeExportReport` must not gain direct host-code materialization behavior.

1.7.5 `kosmo-pipeline`

Cross-layer execution should route through `kosmo-pipeline` or an explicit successor. Policy consistency across subreports is mandatory. A single `Reject` must propagate to the final result.

1.8 Materialization boundary

Phase 11 introduced `MaterializationPlan` and `OperatorApprovalToken`. This remains a governance boundary, not an execution engine.

MATBOUND-001: `MaterializationPlan` is the only allowed entry point for host-write-capable future work.

MATBOUND-002: `OperatorApproved` does not mean execute immediately.

MATBOUND-003: `OperatorApproved` means eligible for Foundry-required materialization.

MATBOUND-004: `FoundryRequired` is still pre-execution.

MATBOUND-005: Actual host mutation requires isolated execution, Foundry, parse-back, evidence capture, and explicit final review.

Part II — Real Foundry and Parse-Back Validation

2.1 Purpose

This part specifies the validation layer required before any real materialization can be considered operational. Foundry proves technical execution. Parse-back proves structural intent.

```
A materialized change is not successful because it was generated.
It is not even successful because it builds.

It is successful only if:
- policy allowed the attempt,
- operator approved the attempt,
- Foundry checks passed,
- parse-back topology matched expectations,
- evidence was captured,
- replay status was preserved.
```

2.2 Foundry runtime boundary

Foundry execution must live under `kosmo-workbench` or a Workbench-owned runtime. HYPHAE, Metatron, LPCM and SystemCube may request validation, but they must not execute validation commands directly.

FOUNDRY-REAL-001: Real Foundry execution **MUST**run through Workbench boundaries.

FOUNDRY-REAL-002: Foundry commands **MUST**be allowlisted by policy.

FOUNDRY-REAL-003: Foundry output **MUST**be captured as evidence refs or digests, not trusted directly.

FOUNDRY-REAL-004: A passing Foundry check is evidence, not promotion.

FOUNDRY-REAL-005: A failed Foundry check **MUST**produce evidence and should create negative evidence.

2.3 FoundryExecutionPlan

```
pub struct FoundryExecutionPlan {
    pub id: Digest,
    pub policy_id: Digest,
    pub workspace_index_id: Digest,
    pub task_id: Digest,
    pub sandbox: FoundrySandboxSpec,
    pub checks: Vec<FoundryCheckSpec>,
    pub command_policy: FoundryCommandPolicy,
    pub timeout_policy: FoundryTimeoutPolicy,
    pub environment_policy: FoundryEnvironmentPolicy,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

2.4 FoundrySandboxSpec

```
pub struct FoundrySandboxSpec {
    pub id: Digest,
    pub sandbox_kind: FoundrySandboxKind,
    pub root_path_digest: Digest,
    pub allow_network: bool,
    pub allow_host_write: bool,
    pub allow_env_passthrough: bool,
    pub writable_paths: Vec<PathDigest>,
    pub readonly_paths: Vec<PathDigest>,
}

pub enum FoundrySandboxKind {
    ReportOnlyNoExec,
    LocalDryRun,
    IsolatedWorktree,
    Container,
}
```

MVP sandbox kinds are `ReportOnlyNoExec`, `LocalDryRun` and `IsolatedWorktree`. Container execution may be deferred.

2.5 FoundryCommandPolicy

```
pub struct FoundryCommandPolicy {
    pub id: Digest,
    pub allowed_commands: Vec<AllowedFoundryCommand>,
    pub forbidden_commands: Vec<String>,
    pub allow_shell: bool,
    pub allow_network_tools: bool,
    pub allow_package_install: bool,
}

pub struct AllowedFoundryCommand {
    pub executable: String,
    pub allowed_args_prefixes: Vec<Vec<String>>,
}
```

Initial allowlist:

```
cargo check
cargo test
cargo fmt --check
cargo clippy --all-targets --all-features -- -D warnings
```

FOUNDRY-CMD-001: Foundry **MUST NOT** execute arbitrary shell strings.

FOUNDRY-CMD-002: Commands **MUST** match an allowlisted executable and argument profile.

FOUNDRY-CMD-003: Network-capable commands **MUST** be forbidden by default.

FOUNDRY-CMD-004: Package installation **MUST** be forbidden by default.

2.6 FoundryExecutionReport

```
pub struct FoundryExecutionReport {
    pub id: Digest,
    pub plan_id: Digest,
    pub policy_id: Digest,
    pub workspace_index_before: Digest,
    pub check_results: Vec<FoundryCheckResult>,
    pub stdout_digest: Option<Digest>,
    pub stderr_digest: Option<Digest>,
    pub output_capture_refs: Vec<EvidenceRef>,
    pub started_at: Timestamp,
    pub finished_at: Timestamp,
    pub duration_ms: u64,
    pub outcome: FoundryExecutionOutcome,
    pub evidence_bundle_id: Digest,
    pub replay_status: ReplayStatus,
}

pub enum FoundryExecutionOutcome {
    SkippedByReportOnly,
    Passed,
    Failed,
    TimedOut,
    CommandDeniedByPolicy,
    SandboxViolation,
    Inconclusive,
}
```

2.7 Parse-back validation

Parse-back validation rescans the result and checks whether the expected topological transformation occurred.

```
after materialization:
workspace rescan
-> WorkspaceIndex after
-> HostCube after
-> CodeHDAG after
-> optional Metatron fingerprints after
-> compare against ParseBackExpectation
```

2.8 ParseBackPlan

```
pub struct ParseBackPlan {
    pub id: Digest,
    pub policy_id: Digest,
    pub materialization_plan_id: Digest,
    pub before_workspace_index_id: Digest,
    pub before_host_cube_id: Digest,
    pub expectations: Vec<ParseBackExpectation>,
    pub scan_profile: ParseBackScanProfile,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct ParseBackScanProfile {
```

```
    pub id: Digest,
    pub include_workspace_rescan: bool,
    pub include_hostcube_rescan: bool,
    pub include_hdag_rescan: bool,
    pub include_metatron_rescan: bool,
    pub max_files: Option<usize>,
    pub max_bytes: Option<u64>,
}
```

2.9 ParseBackExpectation

```
pub struct ParseBackExpectation {
    pub id: Digest,
    pub step_id: Digest,
    pub expectation_kind: ParseBackExpectationKind,
    pub expected_added_voids_resolved: Vec<Digest>,
    pub expected_remaining_voids: Vec<Digest>,
    pub expected_added_roles: Vec<RoleKind>,
    pub expected_removed_antipatterns: Vec<AntiPatternKind>,
    pub expected_added_edges: Vec<CodeEdgeKind>,
    pub expected_removed_edges: Vec<CodeEdgeKind>,
    pub required: bool,
}

pub enum ParseBackExpectationKind {
    VoidResolved,
    EdgeAdded,
    EdgeRemoved,
    RoleAdded,
    AntiPatternRemoved,
    HostTargetDeltaReduced,
    MetatronFingerprintReached,
    SystemCubeDensityChanged,
}
```

2.10 ParseBackReport

```
pub struct ParseBackReport {
    pub id: Digest,
    pub plan_id: Digest,
    pub policy_id: Digest,
    pub after_workspace_index_id: Digest,
    pub after_host_cube_id: Digest,
    pub topology_delta: ParseBackTopologyDelta,
    pub expectation_results: Vec<ParseBackExpectationResult>,
    pub outcome: ParseBackOutcome,
    pub evidence_bundle_id: Digest,
    pub replay_status: ReplayStatus,
}

pub enum ParseBackOutcome {
    Passed,
    PassedWithWarnings,
    FailedTopologyMismatch,
    FailedMissingExpectedChange,
```

```
    FailedResidualAntiPattern,
    Inconclusive,
}
```

2.11 ParseBackTopologyDelta

```
pub struct ParseBackTopologyDelta {
    pub id: Digest,
    pub before_host_cube_id: Digest,
    pub after_host_cube_id: Digest,
    pub resolved_voids: Vec<Digest>,
    pub new_voids: Vec<Digest>,
    pub unchanged_voids: Vec<Digest>,
    pub added_roles: Vec<RoleKind>,
    pub removed_roles: Vec<RoleKind>,
    pub added_edges: Vec<CodeEdgeKind>,
    pub removed_edges: Vec<CodeEdgeKind>,
    pub metatron_before_after: Vec<MetatronFingerprintDelta>,
}
```

2.12 ValidationClosureReport

```
pub struct ValidationClosureReport {
    pub id: Digest,
    pub materialization_plan_id: Digest,
    pub foundry_report_id: Digest,
    pub parseback_report_id: Digest,
    pub foundry_outcome: FoundryExecutionOutcome,
    pub parseback_outcome: ParseBackOutcome,
    pub final_validation_status: ValidationClosureStatus,
    pub evidence_bundle_id: Digest,
}

pub enum ValidationClosureStatus {
    Passed,
    PassedWithWarnings,
    FailedFoundry,
    FailedParseBack,
    FailedBoth,
    Inconclusive,
}
```

VALIDATION-CLOSURE-001: Materialization success requires Foundry pass and required ParseBack pass.

VALIDATION-CLOSURE-002: Foundry pass plus ParseBack fail **MUST NOT**be reported as success.

VALIDATION-CLOSURE-003: Foundry fail plus ParseBack pass is still failure.

VALIDATION-CLOSURE-004: Inconclusive ParseBack **MUST**require human review before promotion.

2.13 Pipeline integration

```
pub fn run_operator_validation_pipeline(
    workspace: &WorkspaceIndex,
    materialization_plan: &MaterializationPlan,
    policy: &PolicyProfile,
) -> Result<ValidationClosureReport, PipelineError>
```

Operational sequence:

1. Verify policy is `OperatorApproved`.
2. Evaluate `MaterializationPlan`.
3. If result is not `FoundryRequired`, stop as blocked.
4. Create isolated worktree.
5. Execute the Workbench task application layer.
6. Run `FoundryExecutionPlan`.
7. Rescan worktree.
8. Run `ParseBackPlan`.
9. Emit `ValidationClosureReport`.
10. Do not auto-merge.

2.14 Acceptance tests

```
FPB-001 ReportOnly policy skips real Foundry execution.
FPB-002 DryRun may execute allowlisted safe command in sandbox.
FPB-003 Non-allowlisted command is denied and produces CommandDeniedByPolicy.
FPB-004 Foundry stdout/stderr are captured as digests/evidence.
FPB-005 MaterializationPlan with valid OperatorApprovalToken returns FoundryRequired,
    not Executed.
FPB-006 Validation pipeline refuses non-OperatorApproved policy.
FPB-007 ParseBackPlan rescans workspace after simulated worktree change.
FPB-008 Expected resolved void passes when after HostCube no longer contains that void
    .
FPB-009 Foundry pass + ParseBack fail yields FailedParseBack.
FPB-010 Foundry fail + ParseBack pass yields FailedFoundry.
FPB-011 Both pass yields ValidationClosureStatus::Passed.
FPB-012 ParseBack mismatch creates negative evidence.
FPB-013 Existing 278-test baseline still passes.
```

Part III — Persistent CorpusCartography Store

3.1 Purpose

This part turns in-memory CorpusCartography into a cross-session structural learning substrate. Persistence enables accumulation, but it must not create authority.

```
Persistence makes HYPHAE cumulative.
Persistence does not make HYPHAE trusted.
```

3.2 CorpusCartographyStore trait

```
pub trait CorpusCartographyStore {
    fn load_scope(
        &self,
        scope: &CorpusScope,
    ) -> Result<CorpusCartography, CartographyStoreError>;

    fn apply_update(
        &self,
        update: CorpusCartographyUpdate,
    ) -> Result<CartographyStoreCommit, CartographyStoreError>;

    fn query(
        &self,
        query: CartographyQuery,
    ) -> Result<CartographyQueryResult, CartographyStoreError>;

    fn verify_scope(
        &self,
        scope: &CorpusScope,
    ) -> Result<CartographyIntegrityReport, CartographyStoreError>;
}
```

3.3 Local append-only store MVP

The first store should be a local deterministic JSON store, readable without network access and easy to inspect.

```
.kosmocrates/
  cartography/
    scopes/
      <scope_digest>/
        manifest.json
        updates/
          0000000000000001_<update_digest>.json
          0000000000000002_<update_digest>.json
        snapshots/
          <cartography_digest>.json
        evidence_refs/
          <evidence_bundle_digest>.json
        quarantine/
```

CARTO-STORE-001: The MVP store **SHOULD**use deterministic file names derived from sequence numbers and update digests.

CARTO-STORE-002: Store writes **MUST**be atomic: write temporary file, sync when available, then rename.

CARTO-STORE-003: A failed write **MUST NOT**advance the manifest head.

CARTO-STORE-004: The store **MUST**be readable without network access.

3.4 CartographyStorageManifest

```
pub struct CartographyStorageManifest {
    pub id: Digest,
    pub scope: CorpusScope,
    pub scope_digest: Digest,
    pub head_cartography_digest: Digest,
    pub head_sequence: u64,
    pub update_digests: Vec<Digest>,
    pub segment_digests: Vec<Digest>,
    pub snapshot_digests: Vec<Digest>,
    pub replay_root: Digest,
    pub policy_id: Digest,
    pub store_format_version: String,
    pub canonicalization_profile_id: Digest,
}
```

CARTO-MANIFEST-001: The manifest head **MUST**match the last committed update.

CARTO-MANIFEST-002: The manifest **MUST**be content-addressed excluding its own id.

CARTO-MANIFEST-003: Digest mismatch **MUST**fail closed.

CARTO-MANIFEST-004: Snapshots may accelerate loading, but update history remains canonical.

3.5 CartographyStoreCommit

```
pub struct CartographyStoreCommit {
    pub id: Digest,
    pub scope: CorpusScope,
    pub sequence: u64,
    pub update_id: Digest,
    pub before_cartography_digest: Digest,
    pub after_cartography_digest: Digest,
    pub manifest_before_digest: Digest,
    pub manifest_after_digest: Digest,
    pub replay_manifest_id: Digest,
    pub evidence_refs: Vec<EvidenceRef>,
    pub commit_status: CartographyStoreCommitStatus,
}

pub enum CartographyStoreCommitStatus {
    Applied,
    IdempotentAlreadyApplied,
    RejectedBeforeDigestMismatch,
    RejectedReplayMismatch,
```

```
    RejectedPolicyMismatch,
    RejectedIntegrityFailure,
}
```

CARTO-COMMIT-001: `apply_update` **MUST**check that `update.before_digest` matches the store head.

CARTO-COMMIT-002: Reapplying the same update **SHOULD**return `IdempotentAlreadyApplied`.

CARTO-COMMIT-003: Before-digest mismatch **MUST**reject fail-closed.

CARTO-COMMIT-004: Every applied commit **MUST**produce a `CartographyStoreCommit`.

3.6 Scope separation

```
pub enum CorpusScope {
    LocalHostProject(Digest),
    WorkspaceFamily(Digest),
    OrganizationPrivateCorpus(Digest),
    ApprovedMirrorCorpus(Digest),
    PublicOpenSourceCorpus,
    DomainProfile(String),
}
```

CARTO-SCOPE-001: A store **MUST NOT**merge different `CorpusScope`s into the same manifest head.

CARTO-SCOPE-002: `PublicOpenSourceCorpus` **MUST NOT**be queried as authority for private or local scope without explicit policy.

CARTO-SCOPE-003: `OrganizationPrivateCorpus` **MUST NOT**leak into `PublicOpenSourceCorpus`.

CARTO-SCOPE-004: Cross-scope lookup **MUST**return provenance and taint summaries.

3.7 CartographyQuery and result

```
pub struct CartographyQuery {
    pub id: Digest,
    pub scope: CorpusScope,
    pub query_kind: CartographyQueryKind,
    pub host_void_kinds: Vec<HostVoidKind>,
    pub motif_kinds: Vec<MotifKind>,
    pub norm_gene_refs: Vec<NormGeneRef>,
    pub microtopology_hashes: Vec<String>,
    pub include_negative_evidence: bool,
    pub include_dormant: bool,
    pub include_quarantined: bool,
    pub policy_id: Digest,
}

pub enum CartographyQueryKind {
    PriorMotifLookup,
    NegativeEvidenceLookup,
    NormGeneHintLookup,
    StructuralCrystalLookup,
    MicroTopologyLookup,
    ConflictLookup,
```

```
    CollapsePlanHintLookup,
}
```

```
pub struct CartographyQueryResult {
    pub id: Digest,
    pub query_id: Digest,
    pub scope: CorpusScope,
    pub matching_entities: Vec<CorpusEntityRef>,
    pub matching_relations: Vec<CorpusRelationRef>,
    pub matching_negative_evidence: Vec<NegativeEvidenceRef>,
    pub matching_crystals: Vec<StructuralCrystalRef>,
    pub matching_norm_genes: Vec<NormGeneRef>,
    pub matching_microtopologies: Vec<MetatronRegionFingerprintRef>,
    pub taint_summary: TaintSummary,
    pub license_summary: LicenseSummary,
    pub authority_summary: AuthoritySummary,
    pub result_status: CartographyQueryResultStatus,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

CARTO-QUERY-001: Queries **MUST**be policy-scoped.

CARTO-QUERY-002: Results **MUST**identify whether matches are active, dormant, superseded, quarantined or negative.

CARTO-QUERY-003: Results **MAY**influence ranking, but **MUST NOT**bypass GateCascade.

CARTO-QUERY-004: Quarantined results **MUST**be excluded unless explicitly requested and policy allows.

3.8 Integrity verification

```
pub struct CartographyIntegrityReport {
    pub id: Digest,
    pub scope: CorpusScope,
    pub manifest_digest: Digest,
    pub checked_update_count: usize,
    pub checked_snapshot_count: usize,
    pub head_matches_manifest: bool,
    pub replay_chain_valid: bool,
    pub evidence_refs_resolvable: bool,
    pub scope_consistent: bool,
    pub canonicalization_valid: bool,
    pub corrupted_segments: Vec<Digest>,
    pub missing_segments: Vec<Digest>,
    pub status: CartographyIntegrityStatus,
}

pub enum CartographyIntegrityStatus {
    Clean,
    Recoverable,
    ReplayRequired,
    QuarantineRequired,
    Unrecoverable,
}
```

CARTO-INTEGRITY-001: `load_scope` **SHOULD**verify manifest digest, head digest and update chain.

CARTO-INTEGRITY-002: `verify_scope` **MUST**report missing or corrupted segments.

CARTO-INTEGRITY-003: If integrity is not clean or recoverable, query **MUST**fail closed unless degraded read is explicitly allowed.

CARTO-INTEGRITY-004: Recovery **MUST**rebuild from update history, not invent state.

3.9 Snapshots and compaction

Snapshots are performance aids, not truth.

```
pub struct CartographySnapshot {
    pub id: Digest,
    pub scope: CorpusScope,
    pub sequence: u64,
    pub cartography_digest: Digest,
    pub source_update_digest: Digest,
    pub snapshot_kind: CartographySnapshotKind,
    pub replay_root: Digest,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum CartographySnapshotKind {
    Full,
    IndexOnly,
    NegativeEvidenceOnly,
    MicroTopologyOnly,
}
```

MVP retention policy:

```
- keep all updates
- optional full snapshot every N updates
- no destructive compaction
```

CARTO-COMPACT-001: The MVP **MUST NOT**perform destructive compaction.

CARTO-COMPACT-002: Compaction **MAY**create derived index files.

CARTO-COMPACT-003: Derived indexes **MUST**be rebuildable from append-only updates.

CARTO-COMPACT-004: Compaction **MUST**declare replay impact.

3.10 Pipeline integration

```
pub struct PipelineCartographyStoreOptions {
    pub store_enabled: bool,
    pub store_path: Option<PathBuf>,
    pub scope: CorpusScope,
    pub verify_before_apply: bool,
    pub snapshot_after_apply: bool,
}

pub fn run_dry_pipeline_with_store(
    index: &WorkspaceIndex,
```

```
    options: &IntegrationRunOptions,
    policy: &PolicyProfile,
    store_options: &PipelineCartographyStoreOptions,
) -> Result<IntegrationRunReport, PipelineError>
```

PIPELINE-CARTO-001: Store-enabled pipeline **MUST**preserve original behavior when `store_enabled=false`.

PIPELINE-CARTO-002: A failed store commit **MUST**not invalidate diagnostic run, but **MUST**mark persistence failure.

PIPELINE-CARTO-003: Persistence failure **MUST NOT**be reported as learning success.

PIPELINE-CARTO-004: The integration report **MUST**include `CartographyStoreCommit` when persistence is enabled.

3.11 Acceptance tests

```
CST-001 Empty store initializes manifest for a scope.
CST-002 apply_update writes update segment and advances manifest head.
CST-003 Reapplying same update is idempotent.
CST-004 apply_update rejects before_digest mismatch.
CST-005 load_scope reconstructs CorpusCartography from update log.
CST-006 verify_scope detects corrupted update segment.
CST-007 corrupted snapshot is ignored and rebuilt from updates.
CST-008 query negative evidence returns NegativeOnly status.
CST-009 query quarantined without permission excludes result.
CST-010 cross-scope query does not merge private and public scopes.
CST-011 store-enabled pipeline records CartographyStoreCommit.
CST-012 store failure marks persistence_failed but does not claim learning success.
CST-013 all prior 278 baseline tests still pass.
```

Part IV — PSE Bridge and Promotion Boundary

4.1 Purpose

This part defines the bridge between the `kosmo-*` production substrate and the existing `pse-*` epistemic substrate.

```
kosmo-* may propose.
pse-* decides crystallization.
```

PSE-BRIDGE-000: The PSE Bridge converts validated `kosmo-*` artifacts into PSE-compatible observation candidates. It **MUST NOT**let `kosmo-*` crates commit PSE `SemanticCrystals` directly.

4.2 Bridge crate placement

Create a separate crate:

```
crates/kosmo-pse-bridge
```

Dependencies:

```
kosmo-pse-bridge
  depends on:
    kosmo-core
    kosmo-hyphae
    kosmo-systemcube
    kosmo-pipeline
    pse-types
  optionally:
    pse-evidence / pse-core behind feature flags
```

BRIDGE-CRATE-001: `kosmo-hyphae` **MUST NOT**depend directly on `pse-core`.

BRIDGE-CRATE-002: `kosmo-pse-bridge` owns all compile-time coupling between `kosmo-*` and `pse-*`.

BRIDGE-CRATE-003: Bridge functionality **SHOULD**be feature-gated until empirically validated.

4.3 PseBridgeCandidate

```
pub struct PseBridgeCandidate {
    pub id: Digest,
    pub source_artifact_id: Digest,
    pub source_artifact_kind: PseBridgeSourceKind,
    pub candidate_kind: PseBridgeCandidateKind,
    pub evidence_bundle_id: Digest,
    pub gate_trace_id: Digest,
    pub policy_id: Digest,
    pub replay_proof_id: Option<Digest>,
    pub foundry_report_id: Option<Digest>,
    pub parseback_report_id: Option<Digest>,
    pub operator_approval_id: Option<Digest>,
    pub bridge_status: PseBridgeCandidateStatus,
```

```
    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub enum PseBridgeSourceKind {
    StructuralCrystalRecord,
    NegativeEvidenceRecord,
    NormGeneCandidate,
    FoundryOutcome,
    ParseBackOutcome,
    SystemCubeManifest,
    KcubeExportReport,
    CorpusCartographyUpdate,
}

pub enum PseBridgeCandidateKind {
    StructuralObservation,
    NegativeEvidenceObservation,
    NormGeneObservation,
    ValidationOutcomeObservation,
    BlueprintObservation,
    PromotionReviewPacket,
}

pub enum PseBridgeCandidateStatus {
    Draft,
    ReadyForPseIngestion,
    RejectedByBridgePolicy,
    RequiresMoreEvidence,
    RequiresHumanReview,
    SubmittedToPse,
    AcceptedByPse,
    RejectedByPse,
}
```

4.4 PseBridgePolicy

```
pub struct PseBridgePolicy {
    pub id: Digest,
    pub allow_structural_crystal_candidates: bool,
    pub allow_negative_evidence: bool,
    pub allow_norm_gene_candidates: bool,
    pub allow_systemcube_observations: bool,
    pub require_assimilation_certificate: bool,
    pub require_foundry_pass_for_executable_effects: bool,
    pub require_parseback_for_materialized_effects: bool,
    pub require_operator_approval_for_promotion: bool,
    pub min_authority_label: AuthorityLabel,
    pub allowed_taint_ceiling: TaintLabel,
    pub default_status: PseBridgeCandidateStatus,
}
```

PSE-BRIDGE-POLICY-001: Default bridge policy **MUST**be disabled or evidence-only.

PSE-BRIDGE-POLICY-002: Executable or materialized effects **MUST NOT**bridge without Foundry and, where applicable, ParseBack evidence.

PSE-BRIDGE-POLICY-003: NormGeneCandidate bridge does not imply TrustedNorm promotion.

PSE-BRIDGE-POLICY-004: Rejected bridge attempts **MUST**preserve negative evidence.

4.5 Evidence mapping

```
pub struct PseEvidenceAdapterBundle {
    pub id: Digest,
    pub kosmo_evidence_bundle_id: Digest,
    pub pse_evidence_refs: Vec<Digest>,
    pub authority_summary: AuthoritySummary,
    pub taint_summary: TaintSummary,
    pub license_summary: LicenseSummary,
    pub replay_status: ReplayStatus,
    pub policy_id: Digest,
    pub adapter_version: String,
}
```

PSE-EVIDENCE-001: Bridge mapping **MUST**preserve original `EvidenceBundle` IDs.

PSE-EVIDENCE-002: PSE evidence adapters **MUST NOT**strip taint, authority or license labels.

PSE-EVIDENCE-003: Lossy evidence mapping **MUST**mark candidate as human-review or more-evidence required.

PSE-EVIDENCE-004: Mapping failure **MUST NOT**mutate original `kosmo-*` records.

4.6 Bridge packets

```
pub struct StructuralCrystalBridgePacket {
    pub id: Digest,
    pub structural_crystal_record_id: Digest,
    pub assimilation_certificate_id: Digest,
    pub constraint_program_id: Digest,
    pub evidence_adapter_bundle_id: Digest,
    pub replay_proof_id: Digest,
    pub pse_candidate_id: Digest,
}

pub struct NegativeEvidenceBridgePacket {
    pub id: Digest,
    pub negative_evidence_record_id: Digest,
    pub rejection_reason: String,
    pub source_gate_trace_id: Digest,
    pub evidence_adapter_bundle_id: Digest,
    pub pse_candidate_id: Digest,
}

pub struct NormGeneBridgePacket {
    pub id: Digest,
    pub norm_gene_candidate_id: Digest,
    pub fitness_trace_id: Digest,
    pub supporting_crystal_ids: Vec<Digest>,
    pub negative_evidence_ids: Vec<Digest>,
    pub evidence_adapter_bundle_id: Digest,
```

```
    pub pse_candidate_id: Digest,
}
```

```
pub struct ValidationOutcomeBridgePacket {
    pub id: Digest,
    pub foundry_report_id: Digest,
    pub parseback_report_id: Option<Digest>,
    pub validation_closure_report_id: Digest,
    pub outcome: ValidationClosureStatus,
    pub evidence_adapter_bundle_id: Digest,
    pub pse_candidate_id: Digest,
}

pub struct SystemCubeBridgePacket {
    pub id: Digest,
    pub systemcube_manifest_id: Digest,
    pub kcube_export_report_id: Digest,
    pub density_report_id: Digest,
    pub contradiction_energy_report_id: Digest,
    pub compatibility_report_id: Digest,
    pub evidence_adapter_bundle_id: Digest,
    pub pse_candidate_id: Digest,
}
```

PSE-CRYSTAL-BRIDGE-001: A StructuralCrystalRecord **MAY**become a bridge candidate only if its certificate is present and replayable.

PSE-NEG-001: NegativeEvidenceRecord **SHOULD**be bridgeable as negative observation.

PSE-NORM-001: NormGeneCandidate **MAY**bridge as candidate observation, not TrustedNorm.

PSE-VALIDATION-001: Foundry pass **MAY**support promotion but **MUST NOT**cause it.

PSE-SYSTEMCUBE-001: SystemCubeManifest **MAY**bridge as BlueprintObservation only.

4.7 PromotionRequest

```
pub struct PromotionRequest {
    pub id: Digest,
    pub bridge_candidate_id: Digest,
    pub promotion_kind: PromotionKind,
    pub evidence_bundle_ids: Vec<Digest>,
    pub gate_trace_ids: Vec<Digest>,
    pub foundry_report_ids: Vec<Digest>,
    pub parseback_report_ids: Vec<Digest>,
    pub operator_approval_id: Option<Digest>,
    pub requested_by: AuthorityLabel,
    pub policy_id: Digest,
}

pub enum PromotionKind {
    PseObservationOnly,
    PseCandidateCrystal,
    PseNegativeEvidence,
    PsePatternMemoryCandidate,
    PseNormReviewCandidate,
    PseSystemBlueprintCandidate,
```

```
}

pub enum PromotionDecision {
    Reject,
    EvidenceOnly,
    HumanReviewRequired,
    BridgeToPseCandidate,
    AcceptedByPseGovernance,
    RejectedByPseGovernance,
}
```

PROMOTION-001: PromotionRequest **MUST**be explicit.

PROMOTION-002: No automatic promotion from `kosmo-*` to PSE trusted memory is allowed.

PROMOTION-003: Promotion rejection **MUST**preserve evidence and diagnostics.

PROMOTION-004: Accepted PSE outcomes **MUST**link back to source `kosmo-*` artifact IDs.

4.8 Pipeline integration

```
pub struct PseBridgeOptions {
    pub bridge_enabled: bool,
    pub bridge_policy: PseBridgePolicy,
    pub promotion_kind: PromotionKind,
    pub submit_to_pse: bool,
}

pub fn run_pipeline_with_pse_bridge(
    index: &WorkspaceIndex,
    options: &IntegrationRunOptions,
    policy: &PolicyProfile,
    bridge_options: &PseBridgeOptions,
) -> Result<IntegrationRunWithBridgeReport, PipelineError>
```

PIPELINE-BRIDGE-001: PSE bridge is disabled by default.

PIPELINE-BRIDGE-002: Bridge-enabled pipeline **MUST**preserve original dry-run report.

PIPELINE-BRIDGE-003: `submit_to_pse=false` emits packets only.

PIPELINE-BRIDGE-004: `submit_to_pse=true` may submit candidate observations, not committed crystals.

4.9 Acceptance tests

```
PSEB-001 Bridge disabled by default.
PSEB-002 StructuralCrystalRecord without AssimilationCertificate is rejected.
PSEB-003 StructuralCrystalRecord with certificate becomes PseBridgeCandidate, not
    SemanticCrystal.
PSEB-004 NormGeneCandidate bridge does not create TrustedNorm.
PSEB-005 NegativeEvidenceRecord can become NegativeEvidenceObservation.
PSEB-006 Foundry pass + ParseBack fail bridges as failed validation outcome.
PSEB-007 SystemCubeManifest bridges with D-density and contradiction report attached.
```

```
PSEB-008 Bridge packet preserves EvidenceBundle, PolicyProfile, GateTrace and
    ReplayProof IDs.
PSEB-009 kosmo-hyphae has no direct dependency on pse-core.
PSEB-010 Bridge failure does not mutate original kosmo artifact.
PSEB-011 PromotionRequest requires explicit requested_by AuthorityLabel.
PSEB-012 Accepted PSE outcome links back to source kosmo artifact IDs.
PSEB-013 All previous baseline tests still pass.
```

Part V — Isolated Materialization and SystemCube Export

5.1 Purpose

This part defines two controlled transitions from planning to artifact:

1. **Isolated materialization**: Workbench materialization tasks are applied in an isolated worktree.
2. **SystemCube disk export**: a blueprint state is written as a deterministic `.kcube` package.

These are not equivalent. Materialization changes a worktree. SystemCube export packages a blueprint.

5.2 Materialization principle

MATERIALIZATION-EXPORT-000: No planned structural change becomes a host effect merely because it exists, scores well, passes D-density, or has an operator token. Host effects require isolated execution, Foundry validation, ParseBack validation, evidence capture and final review.

5.3 IsolatedWorktreeSpec

```
pub struct IsolatedWorktreeSpec {
    pub id: Digest,
    pub source_workspace_index_id: Digest,
    pub source_root_digest: Digest,
    pub worktree_kind: IsolatedWorktreeKind,
    pub worktree_path_digest: Digest,
    pub allow_host_root_write: bool,
    pub allow_network: bool,
    pub allow_env_passthrough: bool,
    pub writable_paths: Vec<PathDigest>,
    pub readonly_paths: Vec<PathDigest>,
    pub cleanup_policy: WorktreeCleanupPolicy,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum IsolatedWorktreeKind {
    GitWorktree,
    TempDirectoryCopy,
    OverlayFs,
    ContainerVolume,
}
```

MVP: use `GitWorktree` when the repository is Git-backed and `TempDirectoryCopy` otherwise.

WORKTREE-001: Materialization **MUST**occur outside the primary host root.

WORKTREE-002: `allow_host_root_write` **MUST**be false for isolated materialization.

WORKTREE-003: The worktree path **MUST**be recorded by digest and evidence.

WORKTREE-004: The worktree **MUST**be disposable unless preserved by policy.

WORKTREE-005: Network access remains false unless explicitly enabled later.

5.4 MaterializationExecutionPlan

```
pub struct MaterializationExecutionPlan {
    pub id: Digest,
    pub materialization_plan_id: Digest,
    pub operator_approval_token_id: Digest,
    pub policy_id: Digest,
    pub worktree_spec: IsolatedWorktreeSpec,
    pub tasks: Vec<WorkbenchMaterializationTask>,
    pub foundry_execution_plan_id: Digest,
    pub parseback_plan_id: Digest,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

MAT-PLAN-001: MaterializationExecutionPlan **MAY**be created only after Materialization-Plan evaluates to FoundryRequired.

MAT-PLAN-002: Every task **MUST**carry Foundry checks and ParseBack expectations.

MAT-PLAN-003: The execution plan **MUST**be content-addressed and evidence-bound.

MAT-PLAN-004: The execution plan **MUST NOT**contain raw LLM patch text as authority.

5.5 WorkbenchTaskApplication

```
pub struct WorkbenchTaskApplication {
    pub id: Digest,
    pub execution_plan_id: Digest,
    pub task_id: Digest,
    pub worktree_before_digest: Digest,
    pub worktree_after_digest: Digest,
    pub diff_digest: Digest,
    pub diff_evidence_ref: EvidenceRef,
    pub applied_files: Vec<PathDigest>,
    pub rejected_files: Vec<PathDigest>,
    pub outcome: WorkbenchTaskApplicationOutcome,
}

pub enum WorkbenchTaskApplicationOutcome {
    AppliedToIsolatedWorktree,
    RejectedByPolicy,
    RejectedByPathScope,
    RejectedByTaint,
    FailedToApply,
    Noop,
}
```

TASK-APPLY-001: Task application **MUST**write only inside the isolated worktree.

TASK-APPLY-002: Every produced diff **MUST**be captured by digest.

TASK-APPLY-003: Path traversal outside the worktree **MUST**be rejected.

TASK-APPLY-004: Rejected task application **MUST**become evidence.

TASK-APPLY-005: A task that applies cleanly is not successful until Foundry and ParseBack pass.

5.6 MaterializationExecutionReport

```
pub struct MaterializationExecutionReport {
    pub id: Digest,
    pub execution_plan_id: Digest,
    pub policy_id: Digest,
    pub task_applications: Vec<WorkbenchTaskApplication>,
    pub foundry_report_id: Digest,
    pub parseback_report_id: Digest,
    pub validation_closure_report_id: Digest,
    pub worktree_before_digest: Digest,
    pub worktree_after_digest: Option<Digest>,
    pub aggregate_diff_digest: Option<Digest>,
    pub outcome: MaterializationExecutionOutcome,
    pub evidence_bundle_id: Digest,
}

pub enum MaterializationExecutionOutcome {
    Blocked,
    AppliedButFoundryFailed,
    AppliedButParseBackFailed,
    AppliedButValidationInconclusive,
    ValidatedInWorktree,
    ReadyForOperatorReview,
}
```

MAT-REPORT-001: `ValidatedInWorktree` does not auto-merge.

MAT-REPORT-002: `ReadyForOperatorReview` means a human/operator may inspect the diff.

MAT-REPORT-003: Failed states **MUST**preserve diffs and failure evidence.

MAT-REPORT-004: No execution report may claim success without `ValidationClosureReport`.

5.7 Final apply boundary

```
pub struct FinalApplyRequest {
    pub id: Digest,
    pub materialization_execution_report_id: Digest,
    pub operator_approval_token_id: Digest,
    pub target_branch_or_path: String,
    pub required_review_refs: Vec<EvidenceRef>,
    pub policy_id: Digest,
}

pub enum FinalApplyDecision {
    Reject,
    RequireReview,
    AllowManualMergeInstruction,
    ApplyToHostDisabled,
}
```

FINAL-APPLY-001: The MVP **MUST NOT**auto-apply validated worktree diffs to the primary host.

FINAL-APPLY-002: FinalApplyRequest **MAY**emit manual merge instructions.

FINAL-APPLY-003: Automatic host merge is out of scope until rollback and policy layers exist.

5.8 SystemCube disk export

SystemCube export packages blueprint state. It does not materialize code.

5.9 KcubePackage

```
pub struct KcubePackage {
    pub id: Digest,
    pub manifest: SystemCubeManifest,
    pub blueprint_units: Vec<BlueprintUnit>,
    pub contradiction_energy_report: ContradictionEnergyReport,
    pub compatibility_profile_report: CompatibilityProfileReport,
    pub d_density_report: DDensityReport,
    pub replay_descriptor: KcubeReplayDescriptor,
    pub package_format_version: String,
}

pub struct KcubeReplayDescriptor {
    pub id: Digest,
    pub source_systemcube_id: Digest,
    pub source_pipeline_report_id: Option<Digest>,
    pub policy_id: Digest,
    pub manifest_digest: Digest,
    pub unit_digests: Vec<Digest>,
    pub report_digests: Vec<Digest>,
    pub canonicalization_profile_id: Digest,
}
```

KCUBE-PACKAGE-001: KcubePackage **MUST**be content-addressed.

KCUBE-PACKAGE-002: Included units **MUST**be accepted, evidence-bound BlueprintUnits.

KCUBE-PACKAGE-003: RejectedOpaque units **MUST NOT**be included.

KCUBE-PACKAGE-004: Contradiction, compatibility and D-density reports **MUST**travel with the package.

KCUBE-PACKAGE-005: The package **MUST**include replay descriptor.

5.10 KcubeExportPolicy

```
pub struct KcubeExportPolicy {
    pub id: Digest,
    pub allow_disk_export: bool,
    pub allowed_output_roots: Vec<PathDigest>,
    pub require_operator_approval: bool,
    pub require_manifest_roundtrip: bool,
    pub require_no_opaque_units: bool,
    pub allow_archive_file: bool,
    pub allow_directory_export: bool,
    pub max_package_bytes: Option<u64>,
}
```

Default policy:

```
allow_disk_export = false
require_operator_approval = true
require_manifest_roundtrip = true
require_no_opaque_units = true
```

KCUBE-POLICY-001: Disk export **MUST**be policy-gated separately from materialization.

KCUBE-POLICY-002: Disk export **MUST**write only under approved output roots.

KCUBE-POLICY-003: Export policy success **MUST NOT**authorize host materialization.

KCUBE-POLICY-004: D-density saturation **MUST NOT**bypass export policy.

5.11 KcubeWriteReport

```
pub struct KcubeWriteReport {
    pub id: Digest,
    pub package_id: Digest,
    pub policy_id: Digest,
    pub output_root_digest: Digest,
    pub output_path_digest: Digest,
    pub bytes_written: u64,
    pub file_count: usize,
    pub roundtrip_digest: Digest,
    pub roundtrip_ok: bool,
    pub mode: KcubeWriteMode,
    pub evidence_bundle_id: Digest,
}

pub enum KcubeWriteMode {
    BlockedByPolicy,
    DryRunOnly,
    WrittenDirectory,
    WrittenArchive,
    RoundtripFailed,
}
```

KCUBE-WRITE-001: KcubeWriteReport **MUST**be emitted for every export attempt.

KCUBE-WRITE-002: Roundtrip **MUST**deserialize and rehash the package.

KCUBE-WRITE-003: `roundtrip_ok=false` means export failure.

KCUBE-WRITE-004: WrittenArchive or WrittenDirectory still does not imply executable trust.

5.12 Deterministic package layout

```
<output_root>/
  <package_digest>.kcube/
    manifest.json
    replay.json
    reports/
      contradiction_energy.json
      compatibility.json
      d_density.json
```

```
    units/
      <unit_digest>.json
    evidence/
      refs.json
```

KCUBE-LAYOUT-001: File ordering inside package **MUST**be deterministic.

KCUBE-LAYOUT-002: JSON serialization **MUST**use canonical profile.

KCUBE-LAYOUT-003: Archive export, if enabled, **MUST**be deterministic.

KCUBE-LAYOUT-004: Exported package **MUST**be reproducible from identical input.

5.13 Acceptance tests

```
MATX-001 Materialization pipeline rejects ReportOnly policy.
MATX-002 Materialization pipeline rejects OperatorApproved policy without valid token.
MATX-003 Materialization creates isolated worktree, not host mutation.
MATX-004 Task application outside worktree is rejected.
MATX-005 Foundry pass + ParseBack pass yields ValidatedInWorktree.
MATX-006 Foundry pass + ParseBack fail yields AppliedButParseBackFailed.
MATX-007 Foundry fail yields AppliedButFoundryFailed.
MATX-008 ValidatedInWorktree does not auto-merge.
KCX-001 Kcube export denied by default policy.
KCX-002 Kcube export allowed only with KcubeExportPolicy.
KCX-003 Exported package round-trips to identical digest.
KCX-004 RejectedOpaque units are excluded.
KCX-005 D-density 1.0 does not bypass KcubeExportPolicy.
KCX-006 SystemCube export writes package files only, not host files.
KCX-007 All prior 278 baseline tests still pass.
```

Part VI — Controlled Source Acquisition and Empirical Evaluation

6.1 Purpose

This part controls new input and measures real usefulness. The substrate may acquire additional sources only through explicit capability, sandboxing, license detection, secret scanning, taint propagation and evidence capture. The substrate may claim architectural usefulness only through empirical evaluation reports.

6.2 Controlled source acquisition

```
LocalFixtureOnly
-> ApprovedLocalPath
-> ApprovedMirror
-> BoundedArchiveDownload
-> BoundedGitShallowClone
```

Network modes remain disabled by default.

6.3 SourceAcquisitionCapability

```
pub struct SourceAcquisitionCapability {
    pub id: Digest,
    pub policy_id: Digest,
    pub issued_by: AuthorityLabel,
    pub acquisition_mode: SourceAcquisitionMode,
    pub allowed_roots: Vec<PathDigest>,
    pub allowed_domains: Vec<String>,
    pub forbidden_domains: Vec<String>,
    pub max_sources: usize,
    pub max_bytes_total: u64,
    pub max_bytes_per_source: u64,
    pub max_depth: usize,
    pub require_license_detection: bool,
    pub require_secret_scan: bool,
    pub require_taint_label: bool,
    pub expires_at: Option<Timestamp>,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum SourceAcquisitionMode {
    LocalFixtureOnly,
    ApprovedLocalPath,
    ApprovedMirror,
    BoundedArchiveDownload,
    BoundedGitShallowClone,
}
```

ACQ-CAP-001: No acquisition may occur without `SourceAcquisitionCapability`.

ACQ-CAP-002: Network acquisition **MUST**remain disabled by default.

ACQ-CAP-003: Capability expiration **MUST**fail closed.

ACQ-CAP-004: Capability scope **MUST**be no broader than active policy.

ACQ-CAP-005: Agent authority alone **MUST NOT**issue acquisition capability.

6.4 AcquisitionSandbox and AcquiredSource

```
pub struct AcquisitionSandbox {
    pub id: Digest,
    pub capability_id: Digest,
    pub sandbox_root_digest: Digest,
    pub acquired_sources: Vec<AcquiredSourceRef>,
    pub license_reports: Vec<LicenseReportRef>,
    pub secret_scan_reports: Vec<SecretScanReportRef>,
    pub taint_summary: TaintSummary,
    pub evidence_bundle_id: Digest,
}

pub struct AcquiredSource {
    pub id: Digest,
    pub acquisition_mode: SourceAcquisitionMode,
    pub source_uri_digest: Digest,
    pub local_sandbox_path_digest: Digest,
    pub byte_size: u64,
    pub content_digest: Digest,
    pub license_status: LicenseStatus,
    pub taint_label: TaintLabel,
    pub authority_label: AuthorityLabel,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

ACQ-SANDBOX-001: Acquired sources **MUST**be stored in acquisition sandbox, not host workspace.

ACQ-SANDBOX-002: Acquired sources are tainted by default.

ACQ-SANDBOX-003: Acquired sources **MUST NOT**enter `ContextPack` directly.

ACQ-SANDBOX-004: SourceCube construction requires license and secret scan.

ACQ-SANDBOX-005: Acquisition output is evidence, not permission.

6.5 License and secret scan

```
pub struct LicenseReport {
    pub id: Digest,
    pub acquired_source_id: Digest,
    pub detected_license: LicenseStatus,
    pub detection_confidence: Q16,
    pub allowed_for_topology_extraction: bool,
    pub allowed_for_context_pack: bool,
    pub allowed_for_materialization: bool,
    pub diagnostics: Vec<String>,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct SecretScanReport {
    pub id: Digest,
```

```
    pub acquired_source_id: Digest,
    pub secret_findings: Vec<SecretFinding>,
    pub risk_score: Q16,
    pub outcome: SecretScanOutcome,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum SecretScanOutcome {
    Clean,
    FindingsRedacted,
    FindingsBlocked,
    ScanFailed,
}
```

ACQ-LICENSE-001: Unknown license **MUST**block ContextPack and materialization use.

ACQ-LICENSE-002: Topology-only extraction **MAY**be allowed under stricter policy if raw content does not enter prompts or patches.

ACQ-SECRET-001: Secret findings **MUST**quarantine acquired source.

ACQ-SECRET-002: Scan failure **MUST**fail closed.

ACQ-SECRET-003: Redacted sources remain tainted.

6.6 SourceEvidence conversion

```
pub struct SourceEvidenceConversionReport {
    pub id: Digest,
    pub acquired_source_id: Digest,
    pub source_evidence_id: Option<Digest>,
    pub license_report_id: Digest,
    pub secret_scan_report_id: Digest,
    pub conversion_status: SourceEvidenceConversionStatus,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum SourceEvidenceConversionStatus {
    ConvertedForTopologyOnly,
    ConvertedForEvidenceOnly,
    BlockedByLicense,
    BlockedBySecretScan,
    BlockedByPolicy,
    Failed,
}
```

ACQ-CONVERT-001: SourceEvidence conversion **MUST**preserve acquisition provenance.

ACQ-CONVERT-002: TopologyOnly conversion **MUST**strip or avoid raw-code prompt use.

ACQ-CONVERT-003: Blocked sources **SHOULD**create negative or acquisition failure evidence.

ACQ-CONVERT-004: Converted SourceEvidence remains subject to GateCascade.

6.7 Empirical evaluation harness

The existing unit tests prove structural invariants. The evaluation harness proves practical usefulness through canonical scenarios.

6.8 EvaluationScenario

```
pub struct EvaluationScenario {
    pub id: Digest,
    pub name: String,
    pub scenario_kind: EvaluationScenarioKind,
    pub fixture_workspace_id: Digest,
    pub expected_voids: Vec<ExpectedVoid>,
    pub expected_gates: Vec<ExpectedGateOutcome>,
    pub expected_reports: Vec<ExpectedReportArtifact>,
    pub policy_id: Digest,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub enum EvaluationScenarioKind {
    MissingTests,
    WeakLayerBoundary,
    HandlerDirectDbAccess,
    ExternalTaintRejected,
    MetatronAmbiguityDowngrade,
    LpcmMajorityCandidateOnly,
    SystemCubeRoundtrip,
    OperatorApprovedValidation,
    CorpusPersistenceReuse,
    PseBridgeCandidateOnly,
}
```

6.9 Canonical evaluation scenarios

Scenario	Expected behavior
EVAL-001 Missing tests	Rust API route without integration tests produces `MissingTestFiber`, test deficiency and planning-only test fiber suggestion.
EVAL-002 Weak boundary	Handler directly queries database; produces weak boundary or bypass diagnostic; any SurgeryOption remains planning-only.
EVAL-003 External taint	External-tainted source is rejected by ContextPack or GateCascade and produces negative evidence.
EVAL-004 LPCM 51%	Fragment field with local majority emits `CandidateDirection` only; no gate bypass.
EVAL-005 SystemCube roundtrip	Accepted BlueprintUnits yield stable package digest; D-density does not authorize materialization.
EVAL-006 OperatorApproved	Valid token and policy produce `FoundryRequired`; execution waits for Foundry and ParseBack.

6.10 EvaluationRunReport and metrics

```
pub struct EvaluationRunReport {
    pub id: Digest,
    pub scenario_id: Digest,
    pub policy_id: Digest,
    pub pipeline_report_id: Digest,
    pub foundry_report_id: Option<Digest>,
    pub parseback_report_id: Option<Digest>,
    pub cartography_store_commit_id: Option<Digest>,
    pub systemcube_write_report_id: Option<Digest>,
    pub pse_bridge_report_id: Option<Digest>,
    pub outcome: EvaluationOutcome,
    pub metric_values: EvaluationMetrics,
    pub evidence_bundle_id: Digest,
}

pub enum EvaluationOutcome {
    Passed,
    PassedWithWarnings,
    FailedExpectedVoidMissing,
    FailedGateMismatch,
    FailedUnexpectedMutation,
    FailedReplayMismatch,
    FailedFoundry,
    FailedParseBack,
    Inconclusive,
}
```

```
pub struct EvaluationMetrics {
    pub void_detection_precision: Q16,
    pub void_detection_recall: Q16,
    pub false_positive_rate: Q16,
    pub false_enrichment_rate: Q16,
    pub negative_evidence_reuse_rate: Q16,
    pub cartography_hit_rate: Q16,
    pub foundry_pass_rate: Q16,
    pub parseback_fidelity: Q16,
    pub deterministic_replay_rate: Q16,
    pub policy_block_correctness: Q16,
    pub repeated_host_speedup: Q16,
}
```

EVAL-METRIC-001: All metrics **MUST**use `Q16`.

EVAL-METRIC-002: Evaluation metrics **MUST NOT**become policy authority.

EVAL-METRIC-003: A high score **MAY**justify future implementation work, not gate bypass.

EVAL-METRIC-004: Failed scenarios **MUST**be retained as regression fixtures.

6.11 Fixture corpus layout

```
fixtures/
  evaluation/
    missing_tests/
      workspace/
      expected.json
    weak_layer_boundary/
```

```
      workspace/
      expected.json
    external_taint/
      workspace/
      expected.json
    lpcm_majority/
      fragments.json
      expected.json
    systemcube_roundtrip/
      units.json
      expected.json
```

6.12 EvaluationHarness

```
pub trait EvaluationHarness {
    fn load_scenario(
        &self,
        scenario_path: &Path,
    ) -> Result<EvaluationScenario, EvaluationError>;

    fn run_scenario(
        &self,
        scenario: &EvaluationScenario,
        policy: &PolicyProfile,
    ) -> Result<EvaluationRunReport, EvaluationError>;

    fn run_suite(
        &self,
        suite: EvaluationSuite,
        policy: &PolicyProfile,
    ) -> Result<EvaluationSuiteReport, EvaluationError>;
}

pub struct EvaluationSuiteReport {
    pub id: Digest,
    pub suite_name: String,
    pub scenario_reports: Vec<EvaluationRunReport>,
    pub aggregate_metrics: EvaluationMetrics,
    pub failed_scenarios: Vec<Digest>,
    pub evidence_bundle_id: Digest,
}
```

6.13 Evaluation gates for future work

EVAL-GATE-001: Real Foundry is incomplete until evaluation proves allowed command execution and denied command rejection.

EVAL-GATE-002: Corpus Persistence is incomplete until cross-run reuse is demonstrated.

EVAL-GATE-003: Materialization Sandbox is incomplete until Foundry plus ParseBack validation is demonstrated in an isolated worktree.

EVAL-GATE-004: SystemCube Export is incomplete until roundtrip export is demonstrated.

EVAL-GATE-005: PSE Bridge is incomplete until candidate-only bridge behavior is demonstrated.

6.14 Acceptance tests

```
ACQ-001 No acquisition without SourceAcquisitionCapability.
ACQ-002 Default policy denies network acquisition.
ACQ-003 ApprovedLocalPath acquisition stores source only in sandbox.
ACQ-004 Unknown license blocks ContextPack and materialization use.
ACQ-005 Secret finding quarantines acquired source.
ACQ-006 Blocked acquisition creates evidence.
ACQ-007 Converted SourceEvidence preserves provenance and taint.
EVAL-001 Missing tests fixture produces MissingTestFiber.
EVAL-002 Weak layer fixture produces boundary/bypass diagnostic.
EVAL-003 External-tainted fixture is rejected by ContextPack/GateCascade.
EVAL-004 LPCM 51% fixture emits CandidateDirection only.
EVAL-005 SystemCube fixture roundtrips deterministically.
EVAL-006 OperatorApproved scenario stops at FoundryRequired unless Foundry+ParseBack
    exist.
EVAL-007 Evaluation metrics use Q16 only.
EVAL-008 Evaluation report is content-addressed and evidence-bound.
EVAL-009 All prior 278 baseline tests still pass.
```

Part VII — Operational Roadmap

7.1 Purpose

This part gives the coding agent the implementation staircase. Each phase must preserve the baseline, add tests, update status files and stop for review.

7.2 R0 — Baseline Lock

```
Run all existing tests.
Record current branch.
Record public API surface.
Refuse implementation if baseline tests fail.
```

Commands:

```
cargo test -p kosmo-core
cargo test -p kosmo-workbench
cargo test -p kosmo-hyphae
cargo test -p kosmo-systemcube
cargo test -p kosmo-pipeline
```

7.3 R1 — Real Foundry MVP

```
- Implement FoundryExecutionPlan.
- Implement FoundrySandboxSpec.
- Implement FoundryCommandPolicy.
- Execute allowlisted local commands.
- Capture stdout/stderr as digests/evidence.
- Deny arbitrary shell strings.
- Preserve ReportOnly skip behavior.
```

7.4 R2 — Parse-Back MVP

```
- Implement ParseBackPlan.
- Implement ParseBackReport.
- Rescan workspace after simulated or isolated worktree change.
- Compare expected resolved voids and topology deltas.
- Emit negative evidence on mismatch.
```

7.5 R3 — Validation Closure

```
- Implement ValidationClosureReport.
- Combine FoundryExecutionReport and ParseBackReport.
- Success requires Foundry pass and required ParseBack pass.
- Foundry pass + ParseBack fail is failure.
```

7.6 R4 — Corpus Store MVP

```
- Implement CorpusCartographyStore trait.
- Implement LocalCorpusCartographyStore.
- Add manifest, update log, snapshots, integrity verify.
- Add negative evidence query.
- Integrate optional store into pipeline.
```

7.7 R5 — Isolated Worktree Materialization MVP

```
- Create isolated worktree.
- Apply WorkbenchMaterializationTask.
- Capture diff.
- Run Foundry.
- Run ParseBack.
- Emit MaterializationExecutionReport.
- No auto-merge.
```

7.8 R6 — SystemCube Export MVP

```
- Build KcubePackage.
- Add KcubeExportPolicy.
- Write deterministic directory package.
- Roundtrip verify.
- Emit KcubeWriteReport.
```

7.9 R7 — PSE Bridge MVP

```
- Add kosmo-pse-bridge crate.
- Emit PseBridgeCandidate.
- Map evidence without stripping taint/authority/license.
- Do not commit PSE SemanticCrystal.
- Bridge disabled by default.
```

7.10 R8 — Controlled Local Acquisition MVP

```
- SourceAcquisitionCapability.
- ApprovedLocalPath only.
- Sandbox only.
- Taint by default.
- License/secret scan stubs fail closed.
- SourceEvidenceConversionReport.
```

7.11 R9 — Evaluation Harness MVP

```
- EvaluationScenario.
- EvaluationRunReport.
- EvaluationMetrics.
- Fixture corpus.
```

```
- EvaluationHarness.
- CI-compatible failure behavior.
```

Part VIII — Coding Agent Handoff

8.1 Required agent behavior

1. Read `SUBSTRATE.md` first.
2. Read `PHASE_CHECKLIST.md` second.
3. Read `IMPLEMENTATION_STATUS.md` third.
4. Inspect existing crates before coding.
5. Run baseline tests before changing code.
6. Implement only the current operational phase.
7. Add tests for success and blocked/failure paths.
8. Update status and traceability documents.
9. Stop after each phase.

8.2 Forbidden actions

```
Do not rebuild kosmo-core.
Do not weaken PolicyProfile.
Do not remove ReportOnly default.
Do not bypass MaterializationPlan.
Do not make SystemCube export executable.
Do not make NormGene trusted.
Do not make CorpusCartography trusted memory.
Do not create PSE SemanticCrystals from kosmo-* directly.
Do not add network acquisition by default.
Do not auto-merge worktree changes.
```

8.3 Control files

The agent must update or create successors to:

```
IMPLEMENTATION_STATUS.md
SPEC_TRACEABILITY.md
IMPLEMENTATION_DECISIONS.md
SAFETY_POLICY.md
PHASE_CHECKLIST.md
SUBSTRATE.md
```

8.4 Response format

After each implementation step, the coding agent must report:

```
Phase:
Files inspected:
Files changed:
Tests run:
Result:
Policy impact:
```

```
Evidence impact:
Regression status:
Open risks:
Next step:
```

Part IX — Acceptance Test Catalog

9.1 Foundry and ParseBack

```
FPB-001 ReportOnly skips Foundry.
FPB-002 DryRun executes allowlisted command.
FPB-003 Disallowed command denied.
FPB-004 stdout/stderr captured as digest.
FPB-005 Foundry pass + ParseBack fail is failure.
FPB-006 Foundry fail + ParseBack pass is failure.
FPB-007 Both pass yields ValidationClosureStatus::Passed.
```

9.2 Corpus Store

```
CST-001 Store initializes scope manifest.
CST-002 apply_update advances head.
CST-003 same update idempotent.
CST-004 before_digest mismatch rejected.
CST-005 load_scope reconstructs state.
CST-006 corrupted update detected.
CST-007 query negative evidence returns NegativeOnly.
```

9.3 PSE Bridge

```
PSEB-001 Bridge disabled by default.
PSEB-002 StructuralCrystal without certificate rejected.
PSEB-003 Candidate is not SemanticCrystal.
PSEB-004 NormGeneCandidate is not TrustedNorm.
PSEB-005 NegativeEvidenceRecord becomes negative observation candidate.
```

9.4 Materialization and export

```
MATX-001 Reject ReportOnly.
MATX-002 Reject missing token.
MATX-003 Use isolated worktree.
MATX-004 No auto-merge.
MATX-005 ValidatedInWorktree only after Foundry + ParseBack.
KCX-001 Export denied by default.
KCX-002 Roundtrip digest stable.
KCX-003 D-density does not bypass export policy.
```

9.5 Acquisition and evaluation

```
ACQ-001 No acquisition without capability.
ACQ-002 Network denied by default.
ACQ-003 ApprovedLocalPath sandboxed.
ACQ-004 Unknown license blocks ContextPack/materialization.
EVAL-001 Missing tests fixture detected.
EVAL-002 External taint rejected.
EVAL-003 Metrics use Q16.
```

```
EVAL-004 Evaluation report content-addressed and evidence-bound.
```

Part X — Definition of Done

10.1 Operationalization DoD

A continuation phase is complete only if:

1. It extends existing `kosmo-*` crates or adds an explicit bridge crate.
2. It does not reimplement completed baseline primitives.
3. It preserves all baseline tests.
4. It adds new tests for success and fail-closed paths.
5. It remains content-addressed.
6. It remains policy-scoped.
7. It emits evidence.
8. It preserves `ReportOnly` default.
9. It does not auto-promote trust.
10. It updates implementation status and traceability documents.

10.2 Full KOSMO-OPS-01 DoD

KOSMO-OPS-01 is complete when:

```
- Real Foundry exists.
- Parse-Back exists.
- CorpusCartography persists across sessions.
- Isolated materialization works but does not auto-merge.
- SystemCube writes deterministic .kcube packages.
- PSE Bridge emits candidates only.
- Controlled local acquisition exists.
- Evaluation harness proves core scenarios.
- All prior baseline tests still pass.
```

Part I — Baseline-to-Operational Delta Table

Area	Current baseline	Operational target
Foundry	Simulated / skipped / skeleton	Real allowlisted sandbox execution with evidence capture
ParseBack	Expectation structures	Rescan + topology delta comparison
CorpusCartography	In-memory append-only map	Persistent append-only scoped store
Materialization	Governance skeleton, FoundryRequired	Isolated worktree application + validation
SystemCube	Dry-run export report	Deterministic `.kcube` package export
PSE Bridge	Deferred conceptual integration	Candidate-only bridge crate
Acquisition	Disabled by default	Local approved sandbox acquisition
Evaluation	Unit/structural tests	Fixture-driven empirical harness
Promotion	No trusted path	Explicit PSE-governed promotion request

Part II — CLI Surface

```
# Foundry / ParseBack
kosmo foundry run --plan <foundry-plan-id>
kosmo parseback run --plan <parseback-plan-id>
kosmo validation close --foundry <id> --parseback <id>

# Corpus store
kosmo cartography init --scope local --path .kosmocrates/cartography
kosmo cartography verify --scope local
kosmo cartography inspect --scope local
kosmo cartography query --scope local --negative-evidence
kosmo pipeline dry-run --with-cartography-store --scope local

# PSE bridge
kosmo pse-bridge candidates --from-run <integration-report-id>
kosmo pse-bridge emit --candidate <id> --dry-run
kosmo pse-bridge promote --candidate <id> --kind observation-only

# Materialization
kosmo materialize plan --from-collapse <plan-id>
kosmo materialize run --plan <materialization-plan-id> --worktree isolated
kosmo materialize status <execution-report-id>
kosmo materialize diff <execution-report-id>

# SystemCube
kosmo systemcube package --from-report <pipeline-report-id> --dry-run
kosmo systemcube export --package <package-id> --out .kosmocrates/exports
kosmo systemcube verify <package-path>

# Acquisition / Evaluation
```

```
kosmo acquire local --capability <cap-id> --path <source-path>
kosmo eval list
kosmo eval run missing_tests --policy report-only
kosmo eval run-suite core --policy report-only
```

Part III — Error Catalog

Error	Meaning
`PolicyDenied`	Active policy does not allow the requested operation.
`DigestMismatch`	Stored digest does not match canonical content.
`BeforeDigestMismatch`	Append-only update does not start from current head.
`CommandDeniedByPolicy`	Foundry command failed allowlist check.
`SandboxViolation`	Execution attempted to leave permitted sandbox.
`ParseBackMismatch`	Expected topology was not observed after rescan.
`BridgePolicyRejected`	PSE bridge policy rejects candidate.
`RoundtripFailed`	Exported package did not rehash to expected digest.
`AcquisitionCapabilityMissing`	Acquisition attempted without explicit capability.
`SecretScanFailedClosed`	Secret scan failed and source was blocked.
`LicenseUnknownBlocked`	Unknown license blocked context/materialization use.

Part IV — Coding Agent Starter Prompt

```
You are implementing KOSMO-OPS-01 in the Kosmocrates repository.

Do not rebuild the existing kosmo-* production substrate.
Start from the current crates:
- kosmo-core
- kosmo-workbench
- kosmo-hyphae
- kosmo-systemcube
- kosmo-pipeline

First read:
- SUBSTRATE.md
- PHASE_CHECKLIST.md
- IMPLEMENTATION_STATUS.md
- SPEC_TRACEABILITY.md
- SAFETY_POLICY.md

Run the baseline tests before changing code.

Implement the current phase only. Add tests for both success and blocked/failure paths
    .
Preserve ReportOnly default, Q16 audit paths, content addressing, evidence binding,
worst-wins gate aggregation, deterministic replay, no network by default, and no
host write without OperatorApproved + token + Foundry + ParseBack.

After each phase, update status and stop.
```